



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

COMPUTER SCIENCE AND ENGINEERING
INGEGNERIA INFORMATICA

Progetto Reti Logiche

2024-25

Autori:

Domenicali Tommaso

Falco Antonio

Codice persona: 10897754, 10920574

Matricola: 213906, 215666

Indice

Indice	2
1 Introduzione	1
1.1 Descrizione Funzionamento	1
1.1.1 Funzionamento del filtro	2
1.1.2 Comportamenti Particolari	3
1.1.3 Memoria	3
1.2 Esempio	5
1.2.1 Parametri di Configurazione	5
1.2.2 Sequenza di dati W	5
1.2.3 Calcolo del Filtraggio	5
1.2.4 Normalizzazione	6
2 Architettura	7
2.1 Descrizione del funzionamento	7
2.2 Segnali interni	9
2.2.1 Segnali di gestione	9
2.2.2 Segnali per il controllo degli stati	9
2.2.3 Segnali per la comunicazione con la memoria	9
2.2.4 Segnali di configurazione e parametri	10
2.3 Descrizione degli Stati e dei Processi	11
2.3.1 Tabella riassuntiva degli stati	14
2.4 Architettura globale	15
3 Risultati Sperimentali	16
3.1 Sintesi	16
3.1.1 Report Timing	16
3.1.2 Report Utilization	17
3.2 Simulazioni	17
3.2.1 Start iniziale	17
3.2.2 Selezione filtro	18
3.2.3 Segnale di reset nel mezzo dell'esecuzione	20
3.2.4 O_done	21
3.2.5 Test con soli valori minimi	23
3.2.6 Test con soli valori massimi	23
3.2.7 Test con sequenza di lunghezza massima	24
4 Conclusioni	25

Elenco delle figure	26
Elenco delle tabelle	27

1 | Introduzione

Il progetto di **Prova Finale** del corso di *Reti Logiche* per l'anno accademico 2024-25 ha come obiettivo la progettazione di un modulo hardware descritto in linguaggio VHDL, in grado di interfacciarsi con una memoria e di elaborare una sequenza di dati secondo le specifiche tecniche fornite.

1.1. Descrizione Funzionamento

Il sistema da realizzare deve essere in grado di leggere dalla memoria una sequenza di valori numerici, elaborarli applicando un filtro differenziale (il cui ordine può essere pari a 3 oppure a 5, in funzione di un apposito parametro di configurazione) e, infine, scrivere nuovamente in memoria la sequenza risultante. La sequenza è costituita da K parole $W + 17$ byte, organizzati nel modo seguente (vedi Figura 1.1):

- $\mathbf{K_1, K_2}$: combinati tra loro rappresentano il numero di valori da elaborare. Il valore K viene ottenuto concatenando K_1 e K_2 così da formare un unico numero.
- $\mathbf{S}$: di cui il bit meno significativo indica l'ordine del filtro da utilizzare, ossia ordine 3 se il bit è pari a 0, ordine 5 se il bit è pari a 1.
- $\mathbf{C_1, \dots, C_{14}}$: coefficienti dei filtri di ordine 3 e 5, rappresentati in complemento a 2.
- $\mathbf{W_1, \dots, W_K}$: valori numerici in ingresso da elaborare, rappresentati in complemento a 2.
- $\mathbf{R_1, \dots, R_K}$: risultati dell'elaborazione, normalizzati e saturati all'intervallo $\langle -128, 127 \rangle$, ovvero posti a -128 se inferiori a tale valore o a 127 se superiori, rappresentati in complemento a 2.

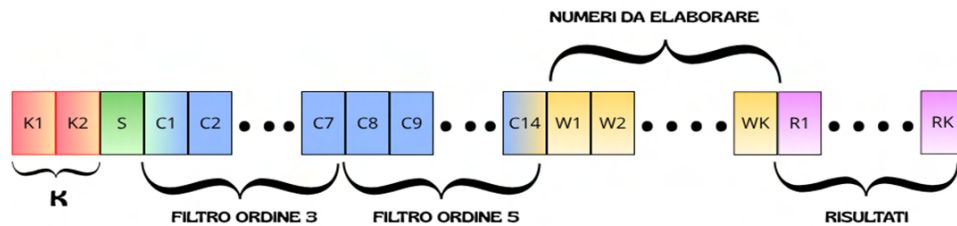


Figura 1.1: Schema rappresentativo della sequenza

Nota

Si osservi che il valore massimo di K può essere pari a 32759 a condizione che $i_add = 0$, poiché il parametro di accesso alla memoria (o_mem_addr) utilizzato dai testbench è a 16 bit ed è impiegato sia nella fase di lettura sia in quella di scrittura in memoria. (Vedi figura 3.9).

Il modulo deve gestire in modo autonomo l'intero processo di lettura, elaborazione e scrittura, operando in modo completamente sincrono rispetto al segnale di clock. Il controllo delle varie fasi operative è affidato a tre segnali principali:

- **RESET**, segnale asincrono che inizializza il sistema e riporta la macchina nello stato di partenza.
- **CLOCK**, segnale di clock generato dal Test Bench. Il modulo lavorerà esclusivamente sul fronte di salite del clock.
- **START**, che avvia il processo di elaborazione.
- **DONE**, che segnala il completamento delle operazioni e la disponibilità dei risultati in memoria.

Durante l'elaborazione, il modulo legge i dati dalla memoria a partire dall'indirizzo iniziale (i_add) della sequenza, acquisisce i valori K_1 , K_2 ed S , carica i coefficienti del filtro, esegue le operazioni di moltiplicazione e somma secondo la funzione riportata in (1.1) e, infine, normalizza i risultati utilizzando approssimazioni binarie della divisione.

La specifica tecnica fornita descrive in maniera dettagliata:

- la **struttura dei dati memorizzati**, comprendente i parametri di configurazione, coefficienti e valori da elaborare.
- la **descrizione funzionale del filtro**, inclusi i criteri di normalizzazione e le approssimazioni necessarie per ottenere risultati corretti anche in presenza di troncamenti.
- le **modalità di interfacciamento con la memoria**, il protocollo di lettura/scrittura e il comportamento temporale dei segnali di controllo.

1.1.1. Funzionamento del filtro

Il filtro ha la seguente funzione:

$$f'(i) = (1/n) * \sum C_j * f[j + i] \quad (1.1)$$

dove:

- **i**: i-esima parola W da elaborare
- **j**: che varia da 2 (filtro 3) a 3 (filtro 5).
- **n**: è il valore di normalizzazione (12 per il filtro 3, 60 per il filtro 5).
- **C_j** : è il j-esimo coefficiente del filtro.
- **$f[i + j]$** : è l'($i + j$)-esimo elemento della finestra di parole W considerata.
- **$\sum$** : che va da $i - j$ fino a $i + j$.

Qualora l'indice $i+j$ risulti al di fuori della sequenza di parole W , il corrispondente valore viene posto uguale a 0

1.1.2. Comportamenti Particolari

- **Attesa e inizio elaborazione**

All'avvio, a seguito dell'attivazione del segnale di RESET (istante iniziale), il modulo si porta in uno stato di attesa, in cui non è richiesto un comportamento specifico se non il mantenimento del segnale *DONE* a 0. L'elaborazione ha inizio quando il segnale *START* viene portato a 1 e mantenuto a livello alto per l'intera durata della computazione.

- **Termine della computazione**

Al completamento dell'elaborazione, il modulo segnala alla memoria la fine dell'esecuzione portando il segnale *DONE* a 1 e mantenendolo a tale livello finché il segnale *START* non viene riportato a 0. Solo in questo momento il segnale *DONE* potrà nuovamente tornare a 0.

- **Seconda Elaborazione**

Per avviare una nuova elaborazione, al termine della precedente, il modulo, riportando *DONE* a 0 (vedi Figure 1.2), indica alla memoria di essere pronto per una successiva computazione. In questo caso, l'inizializzazione della nuova elaborazione non richiede un nuovo segnale di *RESET*, ma avviene esclusivamente tramite la riattivazione del segnale *START* (portato a 1).

- **Segnale di RESET nel bel mezzo dell'esecuzione**

Il segnale di *RESET* può essere portato a 1 in qualsiasi momento, nel caso succeda l'elaborazione deve fermarsi e re-inizializzare l'intero modulo. Poi aspettare un nuovo segnale di *START* = 1 per ripartire nell'elaborazione

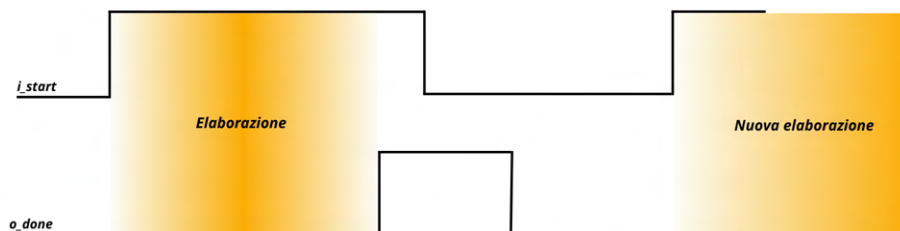


Figura 1.2: Rappresentazione 4-way handshake tra *i_start* e *o_done*

1.1.3. Memoria

All'interno dei testbench è presente un modello comportamentale di memoria che replica il funzionamento di una Block RAM (una memoria RAM interna al dispositivo) sincrona generata tramite Vivado.

Questo modello è derivato direttamente dagli esempi ufficiali riportati nella Vivado Synthesis User Guide (UG901) e riproduce fedelmente il protocollo di accesso adottato dagli strumenti Xilinx.

Caratteristiche dell'interfaccia

- **Lettura sincrona con latenza di 1 ciclo:**

Quando $o_mem_en = '1'$ e $o_mem_we = '0'$, il dato memorizzato all'indirizzo specificato viene reso disponibile sul segnale i_mem_data nel ciclo di clock successivo.

- **Scrittura atomica su singola parola:**

Quando $o_mem_en = '1'$ e $o_mem_we = '1'$, il valore presente su o_mem_data viene scritto nella locazione di memoria indicata da o_mem_addr durante il fronte di salita del clock.

Il componente di memoria è una Block RAM Single-Port configurata in modalità Write-First (se in un ciclo di clock si esegue una scrittura su una cella di memoria, allora subito dopo il fronte di salita del clock, l'uscita **o_mem_data** mostra il dato appena scritto, piuttosto che il valore precedentemente presente in memoria).

Il modello è implementato in stile puramente comportamentale, ma riproduce fedelmente il comportamento hardware della BRAM, assicurando che la simulazione rifletta correttamente un'eventuale architettura reale.

I segnali di interfaccia verso la memoria, pilotati dal modulo, sono riportati in Tabella 1.1.

Porta	Direzione	Funzione
i_clk	in	Clock principale
i_rst	in	Reset asincrono
i_start	in	Avvia l'operazione
i_add	in	Indirizzo base da cui leggere
o_done	out	Segnale che indica la fine
o_mem_addr	out	Indirizzo da inviare alla memoria
i_mem_data	in	Dato letto dalla memoria
o_mem_data	out	Dato da scrivere in memoria
o_mem_we	out	Segnale di scrittura (1) / lettura (0)
o_mem_en	out	Segnale di abilitazione (1) della memoria

Tabella 1.1: Interfaccia del modulo con la memoria

1.2. Esempio

1.2.1. Parametri di Configurazione

Per il Filtro di Ordine 3, i parametri richiesti sono:

- **Lunghezza della sequenza (K):** Minimo 7 byte. Scegliremo $K = 7$ ($K_1 = 00, K_2 = 07$).
- **Selezione Filtro (S):** Il bit meno significativo deve essere 0. Useremo $S = 02$ {hex} (= 00000010 {bin}).
- **Indirizzo di Partenza (ADD):** Scegliremo un indirizzo casuale, ad esempio $ADD = 0100$.
- **Coefficienti del Filtro (Ordine 3):** $c = [0, -1, 8, 0, -8, 1, 0]$ con normalizzazione $n = 12$. L'indice l per la funzione $f'(i)$ è $l = 2$.

La funzione di filtraggio non normalizzata è:

$$\sum = (1) * f[i - 2] + (-8) * f[i - 1] + (0) * f[i] + (8) * f[i + 1] + (-1) * f[i + 2]$$

- **Indirizzo di Inizio Sequenza W (ADDW):** $ADD + 17_{dec} = 0100_{hex} + 11_{hex} = 0111_{hex}$.
- **Indirizzo di Inizio Risultato R (ADDR):** $ADD + 17 + K = 0111_{hex} + 7_{dec} = 0118_{hex}$.

1.2.2. Sequenza di dati W

Definiamo una sequenza di 7 byte W (valori 8-bit in complemento a 2, da -128 a $+127$):

Indice (i)	W_{dec}	W_{hex}	$ADDW_{hex}$
1	10	0A	0111
2	-50	CE	0112
3	120	78	0113
4	-10	F6	0114
5	30	1E	0115
6	-128	80	0116
7	70	46	0117

Tabella 1.2: Sequenza di dati W di esempio

1.2.3. Calcolo del Filtraggio

Ricordando che i valori estremi del filtro sono 0:

i	$f[i-2]$	$f[i-1]$	$f[i]$	$f[i+1]$	$f[i+2]$	$\sum_{\text{dec}}$ non normalizzato	$\sum_{\text{hex}}$
1	0	0	10	-50	120	$(-1) \cdot 0 + 8 \cdot 0 + 0 \cdot 10 + (-8) \cdot (-50) + 1 \cdot 120 = 520$	0208
2	0	10	-50	120	-10	$(-1) \cdot 0 + 8 \cdot 10 + 0 \cdot (-50) + (-8) \cdot 120 + 1 \cdot (-10) = -890$	FC86
3	10	-50	120	-10	30	$(-1) \cdot 10 + 8 \cdot (-50) + 0 \cdot 120 + (-8) \cdot (-10) + 1 \cdot 30 = -300$	FED4
4	-50	120	-10	30	-128	$(-1) \cdot (-50) + 8 \cdot 120 + 0 \cdot (-10) + (-8) \cdot 30 + 1 \cdot (-128) = 642$	0282
5	120	-10	30	-128	70	$(-1) \cdot 120 + 8 \cdot (-10) + 0 \cdot 30 + (-8) \cdot (-128) + 1 \cdot 70 = 894$	037E
6	-10	30	-128	70	0	$(-1) \cdot (-10) + 8 \cdot 30 + 0 \cdot (-128) + (-8) \cdot 70 + 1 \cdot 0 = -310$	FECA
7	30	-128	70	0	0	$(-1) \cdot 30 + 8 \cdot (-128) + 0 \cdot 70 + (-8) \cdot 0 + 1 \cdot 0 = -1054$	FBE2

Tabella 1.3: Calcolo del filtraggio con i dati d'esempio.

1.2.4. Normalizzazione

La costante di normalizzazione del filtro di ordine 3 è pari a $\frac{1}{12}$, approssimata come somma di potenze negative di due:

$$\frac{1}{12} \approx \frac{1}{16} + \frac{1}{64} + \frac{1}{256} + \frac{1}{1024}$$

Per i valori negativi, al risultato di ciascuna operazione di *shift* viene aggiunta un'unità per compensare l'errore dovuto alla troncatura.

- Filtro utilizzato: ordine 3 $\rightarrow$ *coefficienti* = $[0, -1, 8, 0, -8, 1, 0]$

i	$\sum$ (dec)	Calcolo normalizzazione (dec)	R sat. $[-128, 127]$	R_{hex}	ADDR $_{\text{hex}}$
1	520	$32 + 8 + 2 + 0$	42	2A	0118
2	-890	$-55 - 13 - 3 + 0$	-71	B9	0119
3	-300	$-18 - 4 - 1 + 0$	-23	E9	011A
4	642	$40 + 10 + 2 + 0$	52	34	011B
5	894	$55 + 13 + 3 + 0$	71	47	011C
6	-310	$-19 - 4 - 1 + 0$	-24	E8	011D
7	-1054	$-65 - 16 - 4 - 1$	-86	AA	011E

Tabella 1.4: Calcolo della normalizzazione sui dati di esempio.

2 | Architettura

2.1. Descrizione del funzionamento

L'implementazione del modulo si basa su un'architettura di controllo sequenziale (macchina a stati finiti, FSM) progettata per sincronizzare le operazioni di lettura dalla memoria, elaborazione dei calcoli e scrittura dei risultati. Il sistema gestisce le latenze della memoria introducendo opportune fasi di attesa, in modo da garantire la stabilità dei dati prima del loro utilizzo.

Le operazioni possono essere suddivise in cinque macro-fasi principali come segue:

1. Acquisizione dei parametri di configurazione

- **Lettura della lunghezza della sequenza (K)**

Il sistema legge dalla memoria il valore K , che indica il numero di parole W che dovranno essere elaborate. Poiché il bus dati è a 8 bit mentre K è rappresentato su 16 bit, la lettura avviene in due cicli distinti: prima viene acquisita la parte più significativa (K_1) e, successivamente, la parte meno significativa (K_2). Le due parti vengono quindi concatenate in un unico registro interno (K).

- **Configurazione della tipologia di filtro (S)**

Viene letto il parametro S . Sulla base del bit meno significativo di tale valore ($S_{(0)}$), il sistema seleziona il set di coefficienti corrispondente (filtro di ordine 3 oppure filtro di ordine 5). Questa selezione determina automaticamente l'indirizzo di memoria a partire dal quale si inizierà a leggere i coefficienti del filtro (C_1 o C_8 rispettivamente).

- **Caricamento dei coefficienti**

Il modulo esegue una serie di letture sequenziali per caricare i 7 coefficienti necessari all'interno di un registro interno (*coeff_array*), che verrà utilizzato durante la fase di elaborazione.

2. Lettura e gestione dei dati (finestra mobile)

- **Riempimento del buffer di elaborazione**

I dati vengono prelevati dalla memoria e inseriti in un buffer temporaneo composto da 7 elementi. Ad ogni nuovo ciclo di elaborazione, i dati nel buffer verranno poi fatti scorrere di una posizione per lasciare spazio al nuovo dato in ingresso che prenderà posto in ultima posizione.

- **Gestione dei dati a bordo sequenza**

Il sistema monitora costantemente se l'indice di lettura ha raggiunto la fine della lunghezza definita da K . Qualora la finestra di elaborazione richieda campioni oltre la lunghezza della sequenza originale, il sistema inserisce automaticamente

valori nulli (0) nel buffer, consentendo al calcolo di proseguire fino all'ultimo campione senza accessi fuori dai limiti.

3. Elaborazione del filtro

- **Calcolo della somma pesata**

Una volta che i dati nel buffer risultano stabili, il sistema esegue le moltiplicazioni tra ciascun dato e il corrispondente coefficiente del filtro. I prodotti vengono accumulati in un registro temporaneo (*temp_sum*) a precisione estesa (signed a 32 bit), al fine di evitare eventuali overflow dovuti alle somme parziali.

- **Normalizzazione e correzione**

Il risultato della somma viene ora normalizzato per riportarlo alla scala corretta (8 bit, in complemento a 2). Anzichè eseguire una divisione decimale standard, il sistema utilizza una combinazione di scorrimenti di bit (shift) e somme, che approssimano la divisione richiesta in funzione del filtro selezionato. Per i valori negativi viene inoltre applicato un opportuno fattore di correzione (+1 per ogni approssimazione negativa), così da compensare l'errore introdotto dagli shift su numeri con segno.

- **Saturazione del risultato**

Il valore finale viene verificato per garantire che rientri nell'intervallo rappresentabile su 8 bit, ovvero da -128 a $+127$. Qualora il risultato superi tali limiti, esso viene saturato al valore massimo o minimo consentito, prevenendo fenomeni di overflow che comporterebbero un ribaltamento del segno.

4. Scrittura del Risultato

- **Indirizzamento e salvataggio**

Il modulo calcola quindi l'indirizzo di memoria in cui scrivere il risultato filtrato: tale indirizzo è scelto in modo da collocare i dati elaborati in un'area di memoria direttamente successiva a quella contenente i dati di ingresso. Una volta calcolato l'indirizzo corretto, viene richiesto ed attivato il segnale di scrittura per salvare il dato in memoria.

- **Avanzamento del processo**

Al termine della scrittura, il contatore delle parole elaborate viene incrementato. Se non è ancora stato raggiunto il numero totale di parole K , il sistema ritorna alla fase di lettura per elaborare il campione successivo.

5. Terminazione dell'elaborazione

- **Conclusione del ciclo**

Quando tutte le parole sono state processate e salvate, viene attivato il segnale di fine operazione (*o_done*): i registri interni e i contatori vengono riportati ai valori iniziali, ripristinando lo stato del modulo. Il sistema rimane in attesa che il segnale di avvio venga disattivato (*i_start* = '0') prima di tornare nello stato di riposo.

2.2. Segnali interni

I segnali interni, utilizzati per il controllo del flusso di esecuzione e per la gestione dei dati, si suddividono nelle seguenti categorie.

2.2.1. Segnali di gestione

- **Buffer_index**: definito come `integer`, indica la posizione dell'ultimo elemento caricato all'interno di `data_window`.
- **Current_index**: definito come `integer`, indica la posizione corrente nella sequenza raggiunta dalla lettura dei dati.
- **Processing_counter**: definito come `integer`, conta il numero di elaborazioni eseguite, consentendo di determinare quando il modulo ha terminato la scrittura dei risultati.
- **Coeff_counter**: definito come `integer`, indica il numero di coefficienti caricati all'interno di `coefficients`, in modo da stabilire il termine della fase di caricamento.

2.2.2. Segnali per il controllo degli stati

Gli stati della FSM sono raccolti in un tipo enumerato denominato "`state_type`", che definisce tutte le fasi operative del modulo.

```
1  type state_type is (  
2      IDLE ,  
3      READ_K1 , SETTLE_K1 ,  
4      READ_K2 , SETTLE_K2 ,  
5      READ_S ,  SETTLE_S ,  
6      SET_S ,   SETTLE_PRELOAD ,  
7      LOAD_COEFF , SETTLE_COEFF ,  
8      INIT_PROCESSING ,  
9      PROCESSING ,  
10     DONE_STATE  
11 );
```

- **Current_state**: definito come `state_type`, rappresenta lo stato corrente della FSM principale.
- **Next_state**: definito come `state_type`, rappresenta il prossimo stato della FSM; all'inizio di ogni ciclo di clock viene assegnato a `current_state` tramite l'istruzione `current_state <= next_state`.
- **Processing_phase**: definito come `integer`, indica lo stato interno della macchina a stati dedicata al blocco di elaborazione (`Processing`).

2.2.3. Segnali per la comunicazione con la memoria

- **Mem_addr_int**: definito come vettore a 16 bit, viene assegnato al segnale `o_mem_addr`.
- **Mem_data_out_int**: definito come vettore a 8 bit, viene assegnato al segnale `o_mem_data`.

- **Mem_we_int**: definito come *std_logic*, viene assegnato al segnale *o_mem_we*.
- **Mem_en_int**: definito come *std_logic*, viene assegnato al segnale *o_mem_en*.
- **Done_int**: definito come *std_logic*, viene assegnato al segnale *o_done*.

2.2.4. Segnali di configurazione e parametri

Nome	Tipo	Descrizione
K	<i>std_logic_vector</i> (15 downto 0)	Concatenazione di K_1 e K_2 ; rappresenta il numero di parole W da elaborare.
Base_address	<i>std_logic_vector</i> (15 downto 0)	Indirizzo iniziale della sequenza di dati in memoria.
Filter_select	<i>std_logic</i>	Selettore del filtro (0 = filtro di ordine 3, 1 = filtro di ordine 5).
Coefficients	<i>coeff_array</i> (<i>signed</i> (7 downto 0))	Array contenente i coefficienti del filtro.
Data_window	<i>data_buffer</i> (<i>signed</i> (7 downto 0))	Finestra scorrevole di dati da elaborare.
Temp_sum	<i>signed</i> (31 downto 0)	Risultato parziale per fase di elaborazione.
Normalized_result	<i>signed</i> (7 downto 0)	Risultato normalizzato e saturato da scrivere in memoria
Current_index	<i>integer</i>	Indice corrente in memoria.
Coeff_counter	<i>integer</i>	Contatore del numero di coefficienti caricati.
Processing_counter	<i>integer</i>	Contatore delle elaborazioni eseguite (indice del campione corrente).
Processing_phase	<i>integer</i>	Fase corrente della procedura di elaborazione (ad es. 0–9).
Buffer_index	<i>integer</i>	Posizione corrente all'interno del buffer (ad es. 0–6).

Tabella 2.1: Registri e buffer principali del modulo (tutti dichiarati come *signal*).

- **K**: definito come vettore a 16 bit, ottenuto dalla concatenazione di K_1 e K_2 ; indica il numero di parole W da elaborare.
- **Filter_selected**: definito come *std_logic* (1 bit), indica il filtro selezionato in base al valore letto ('0' = filtro di ordine 3, '1' = filtro di ordine 5).
- **Base_address**: definito come vettore a 16 bit, rappresenta l'indirizzo di partenza della sequenza, derivato dal segnale *i_add*.
- **Coefficients**: definito come *coeff_array*, a sua volta un array di 7 byte, utilizzato per memorizzare i coefficienti del filtro letti dalla memoria.
- **Data_window**: definito come *data_buffer*, a sua volta un array di 7 byte, utilizzato come finestra scorrevole sui dati della sequenza da elaborare.
- **Temp_sum**: definito come *signed* a 32 bit, utilizzato per accumulare la somma dei prodotti tra i dati della finestra e i coefficienti del filtro.

- **Normalized_result**: definito come *signed* a 8 bit, utilizzato per memorizzare l'i-esimo risultato dopo la fase di normalizzazione (ed eventuale saturazione).

2.3. Descrizione degli Stati e dei Processi

La macchina a stati finiti è costituita da 14 stati:

- IDLE: attende il segnale di avvio *i_start*.
- READ_K1 / SETTLE_K1: lettura e stabilizzazione del byte alto di *K*.
- READ_K2 / SETTLE_K2: lettura e stabilizzazione del byte basso di *K*.
- READ_S / SETTLE_S: lettura e stabilizzazione del selettore *S*.
- SET_S: selezione dei coefficienti per filtro di ordine 3 o 5.
- SETTLE_PRELOAD: ciclo di attesa per inizializzare i dati.
- LOAD_COEFF / SETTLE_COEFF: caricamento e stabilizzazione dei coefficienti.
- INIT_PROCESSING: preparazione della fase di elaborazione.
- PROCESSING: gestione finestra dati, convoluzione, normalizzazione e scrittura.
- DONE_STATE: completamento del modulo.

Il modulo è controllato da due processi principali:

1. **State_update_process**: Questo processo gestisce l'evoluzione dello stato della FSM. L'inclusione di *i_start* nella sensitivity list consente un reset asincrono più reattivo(vedi Figure 2.1).

Le transizioni particolari dello stato sono:

- IDLE $\rightarrow$ READ_K1 quando *i_start* = '1'.
- SETTLE_COEFF $\rightarrow$ LOAD_COEFF se *coeff_counter* < 6.
- SETTLE_COEFF $\rightarrow$ INIT_PROCESSING se *coeff_counter* $\geq$ 6.
- PROCESSING $\rightarrow$ DONE_STATE quando *processing_counter* $\geq$ *K*. mentre tutte le altre avvengono verso lo stato successivo a ogni fronte di salita del clock, senza ulteriori condizioni interne da soddisfare.

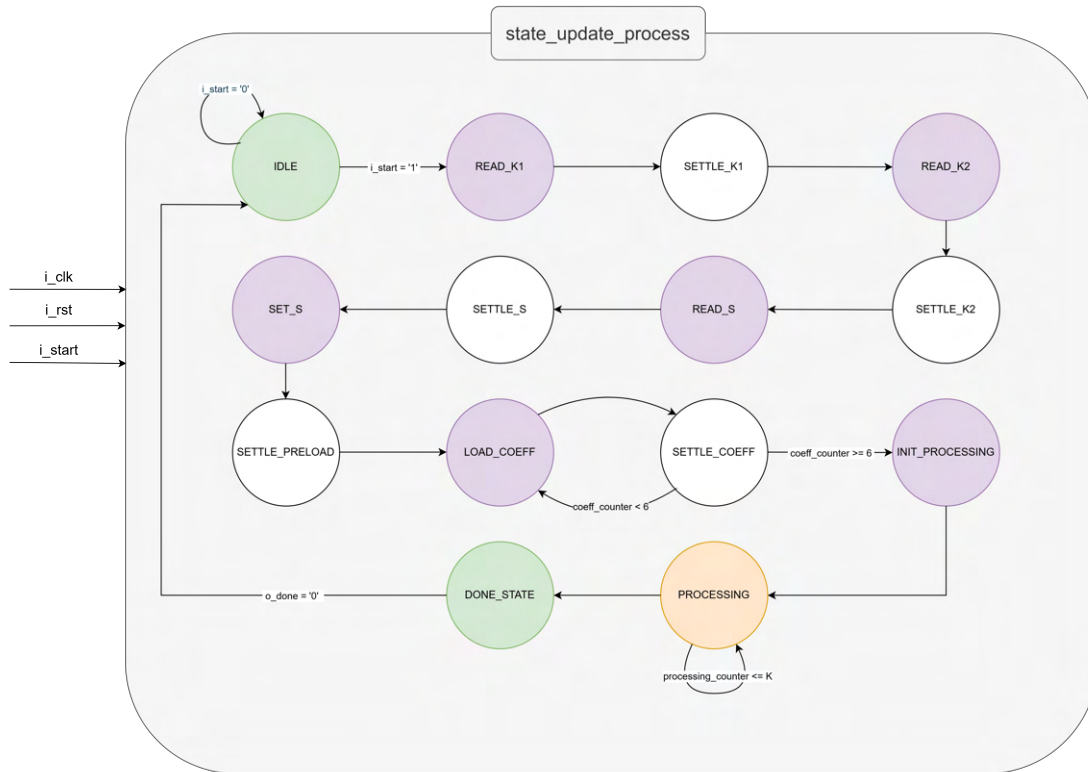


Figura 2.1: Schema della FSM di `state_update_process`

2. **Main_control_process:** Questo processo controlla tutte le operazioni di lettura e scrittura in memoria e la gestione interna delle variabili operative. Il comportamento dipende dallo stato della FSM.

Se $i_rst = '1'$ tutti i segnali vengono resettati e lo stato corrente viene imposto ad IDLE. Altrimenti, sul fronte di salita del clock:

- **IDLE:** prepara lettura, abilita memoria, salva indirizzo di base.
- **READ_K1:** avvia lettura del byte alto di K , incrementa *current_index*.
- **READ_K2:** salva $K(15 : 8)$, avvia lettura di $K(7 : 0)$, incrementa *current_index*.
- **READ_S:** salva $K(7 : 0)$, prepara lettura del selettore S .
- **SET_S:** seleziona tipo di filtro e aggiorna l'indice di lettura rispettivamente di +1 per il filtro di ordine 3, +8 per il filtro di ordine 5.
- **LOAD_COEFF:** inserisce il valore letto dalla memoria (richiesto al ciclo di clock precedente) nel vettore dei coefficienti (la posizione è gestita dalla variabile *coeff_counter*, inizializzata a 0). Successivamente verifica quanti coefficienti sono stati caricati e, se *coeff_counter* < 6, incrementa *coeff_counter* e *current_index* di 1, così da proseguire la lettura iniziando l'indirizzo di memoria per il coefficiente successivo; in caso contrario il caricamento dei coefficienti è terminato e lo stato non esegue ulteriori operazioni.
 - **INIT_PROCESSING:** Innanzitutto verifica il filtro selezionato. Se si tratta di un filtro di ordine 3, imposta il primo e l'ultimo coefficiente a 0 (poiché il filtro utilizza 5 coefficienti invece di 7). Successivamente porta

current_index 17 posizioni avanti rispetto all'indirizzo base (prima parola W_1) e riabilita il segnale di lettura impostando $ENABLE = 1$.

- **PROCESSING (FSM interna):** Quando $processing_counter < K$, la fase di PROCESSING viene gestita da una FSM interna a 10 fasi ($processing_phase = 0, \dots, 9$), che coordina caricamento, calcolo e scrittura del risultato (vedi Figura 2.2). Una volta superato il valore K , il modulo termina le operazioni e disabilita l'accesso alla memoria ($ENABLE = 0$, $WRITE_ENABLE = 0$). Gli stati sono definiti tramite la variabile intera $processing_phase$, inizializzata a 0:
 - * **0:** abilita $ENABLE$, imposta $WRITE_ENABLE = 0$, passa allo stato successivo.
 - * **1:** stato di attesa propagazione, passa allo stato successivo.
 - * **2:** gestione finestra mobile ($data_window$):
 - Se $buffer_index < 7$: fase iniziale del caricamento del buffer. Inserisce il dato letto da memoria nella posizione indicata da $buffer_index$ (inizializzato a 3), aggiorna $current_index$ e $buffer_index$.
Se $buffer_index = 6$, passa allo stato successivo, altrimenti $processing_phase = 1$.
 - Se $buffer_index \geq 7$: il buffer è già pieno. Effettua uno shift a sinistra dei dati e inserisce il nuovo dato in ultima posizione, aggiorna $current_index$ e passa a $processing_phase = 3$.
 - * **3:** calcolo del filtraggio; assegna il risultato a $temp_sum$ e passa allo stato successivo.
 - * **4:** normalizzazione e saturazione del risultato (limiti $[-128, +127]$); passa allo stato successivo.
 - * **5:** calcolo dell'indirizzo di scrittura e preparazione del dato; passa allo stato successivo.
 - * **6:** abilita scrittura in memoria; passa allo stato successivo.
 - * **7:** stato di attesa propagazione; passa allo stato successivo.
 - * **8:** ripristino indirizzo di lettura; se $processing_counter < K$ torna allo stato iniziale ($processing_phase = 0$), altrimenti passa allo stato successivo.
 - * **9 (others):** stato di sicurezza; porta a $DONE_STATE$ e imposta $processing_phase = 1$.
- **DONE_STATE** Tutti i segnali vengono riportati ai valori iniziali e viene attivato $DONE = 1$, rendendo il modulo pronto per eventuali successive operazioni.

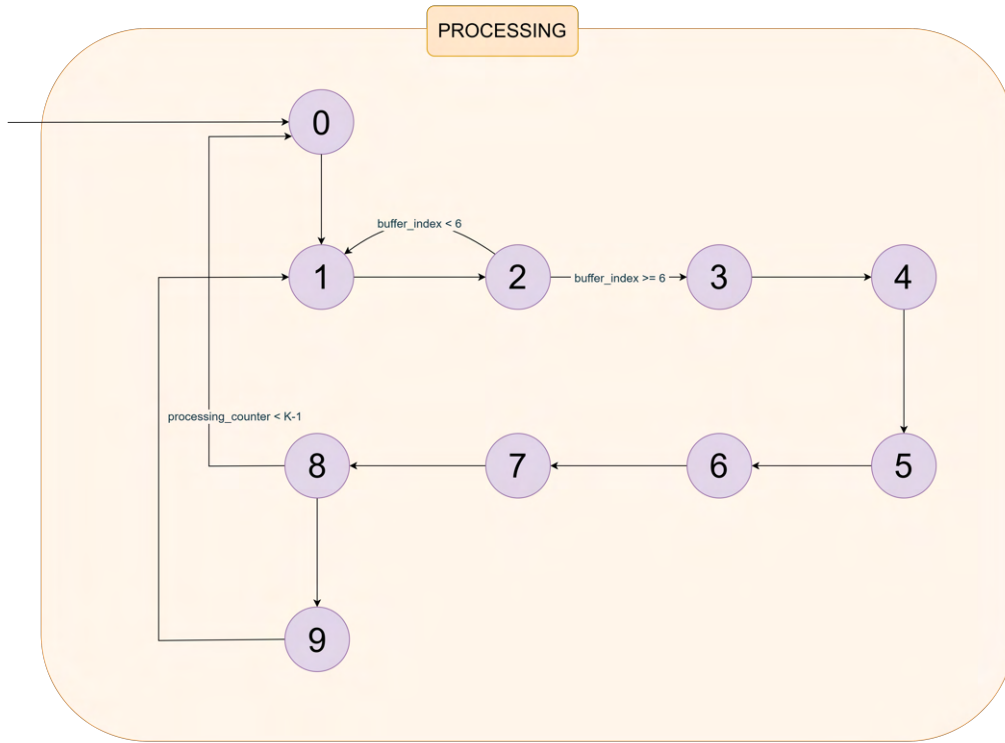


Figura 2.2: Rappresentazione della FSM di *processing_phase*

2.3.1. Tabella riassuntiva degli stati

Stato	Funzione	Cicli	Operazioni
IDLE	Sleep state	–	Attende $i_start = 1$, salva indirizzo di base.
READ_K1	Lettura K (MSB)	1	Avvia lettura del byte alto di K .
SETTLE_K1	Stabilizzazione	1	Attesa propagazione.
READ_K2	Lettura K (LSB)	1	Salva $K(15 : 8)$, avvia lettura di $K(7 : 0)$.
SETTLE_K2	Stabilizzazione	1	Attesa propagazione.
READ_S	Lettura selettore S	1	Salva $K(7 : 0)$, avvia lettura di S .
SETTLE_S	Stabilizzazione	1	Attesa propagazione.
SET_S	Configurazione filtro	1	Valuta $S(0)$, prepara eventuale offset coefficienti.
SETTLE_PRELOAD	Stabilizzazione	1	Prepara indirizzamento coefficienti.
LOAD_COEFF	Caricamento coefficienti	7	Carica coefficienti del filtro.
SETTLE_COEFF	Stabilizzazione	7	Attesa propagazione.
INIT_PROCESSING	Inizializzazione	1	Attende ultimo valore del filtro e prepara elaborazione.
PROCESSING	Elaborazione dati e calcoli	$9K$	Itera l'elaborazione per i K campioni (suddiviso in 9 sub-stati ogni ciclo).
DONE_STATE	Completamento	–	$o_done = 1$, attende $i_start = 0$.

Tabella 2.2: Latenza totale = $23 + 9K$ cicli di clock

2.4. Architettura globale

Per facilitare la comprensione visiva della logica di controllo e delle interconnessioni dei dati, è stato realizzato uno schema a blocchi che rappresenta il datapath del modulo, con particolare attenzione alle operazioni del processo sincrono *main_control_process*. Lo scopo del diagramma non è replicare tutte le assegnazioni VHDL, che sono state omesse per chiarezza, ma evidenziare l'utilizzo e il ripristino dei segnali principali del sistema durante la fase di *PROCESSING*.

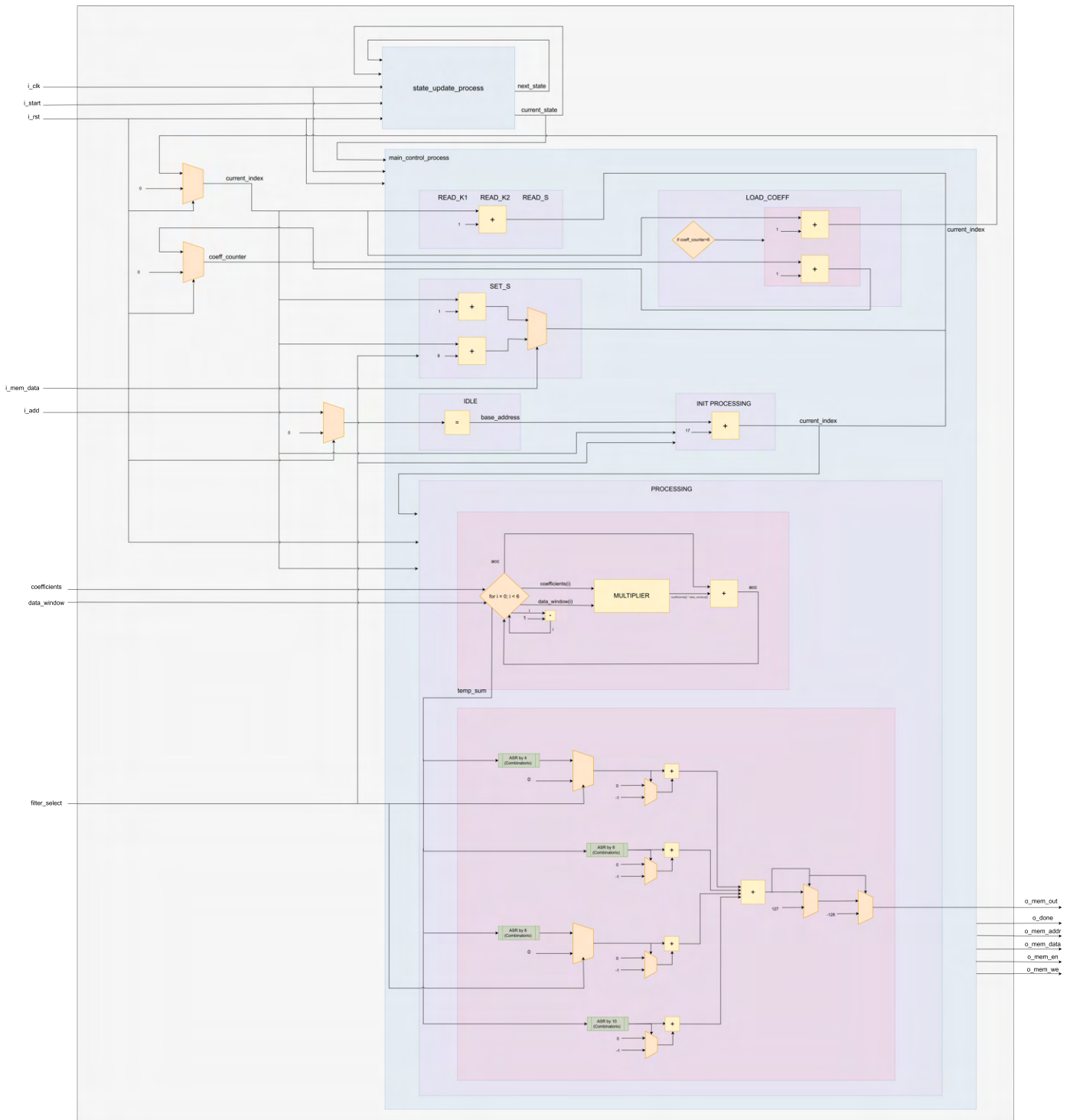


Figura 2.3: Architettura globale

3 | Risultati Sperimentali

3.1. Sintesi

3.1.1. Report Timing

A seguito dell'analisi temporale statica, condotta mediante il comando *report_timing* di Vivado, si evidenzia quanto segue (sintesi in Tabella 3.1):

- Il **Worst Negative Slack (WNS)**, calcolato rispetto a un clock con periodo pari a 20 ns, risulta essere di 10.815 ns. Questo valore indica che il percorso critico del progetto presenta un ritardo complessivo di circa 9.185 ns, ottenuto sottraendo il WNS dal periodo di riferimento.
- Il **marginale temporale disponibile** è quindi molto ampio: il progetto dispone infatti di oltre 10 ns di slack positivo, segno che il vincolo di timing attuale è rispettato con largo vantaggio. Poiché il ritardo del percorso critico è inferiore alla metà del periodo definito, il comportamento temporale del design risulta particolarmente stabile.
- Abbiamo visto che il **percorso critico** attraversa 23 livelli logici ed è associato alle operazioni di somma che contribuiscono alla generazione del segnale *temp_sum*
- Non sono presenti violazioni di Setup nè di Hold.

Parametro	Valore
Slack (MET)	+10.815 ns
Path Type	Setup (Slow Corner)
Source → Dest.	coeff[6][2] → temp_sum[31]
Data Path Delay	9.042 ns
Logic / Routing	4.822 ns / 4.220 ns
Logic Levels	23 (CARRY4+LUT)
Required / Arrival	21.974 ns / -11.159 ns
Slack totale	+10.815 ns

Tabella 3.1: Sintesi del Timing Report

3.1.2. Report Utilization

Per quanto concerne l'occupazione d'area, l'analisi post-implementazione, effettuata tramite il comando `report_utilization`, evidenzia l'assenza di latch nel report di utilizzo (vedi Tabella 3.2) e conferma la correttezza della descrizione VHDL. Diversamente dai flip-flop, i latch sono sensibili al livello del segnale e non al fronte di clock; la loro comparsa involontaria, spesso dovuta a condizioni `if-else` o `case` non completamente definite nei processi combinatori, avrebbe potuto compromettere la stabilità del design, rendendolo vulnerabile e complicando l'analisi temporale statica (STA).

Essendo il design completamente sincrono, il modulo garantisce quindi un comportamento deterministico e affidabile.

Site Type	Used	Fixed	Available	Util(%)
Slice LUTs	1049	0	41000	2.56
LUT as Logic	1049	0	41000	2.56
LUT as Memory	0	0	13400	0.00
Slice Registers	323	0	82000	0.39
Register as Flip Flop	323	0	82000	0.39
Register as Latch	0	0	82000	0.00
F7 Muxes	0	0	20500	0.00
F8 Muxes	0	0	10250	0.00

Tabella 3.2: Report di utilizzo delle risorse

3.2. Simulazioni

3.2.1. Start iniziale

Dopo il reset, il modulo si trova in IDLE: il segnale di completamento è basso ($o_done = 0$) e tutti i contatori interni sono azzerati. Il modulo rimane in questo stato di attesa indefinitamente, aspettando il segnale i_start si alzi: questo è reso possibile dall'inclusione nella sensitivity list di `next_state_logic` di i_start . Quando il segnale i_start viene alzato a '1', il modulo rileva immediatamente questa condizione. Nel processo di controllo principale, eseguito sul fronte di salita del clock, viene eseguito il case statement sullo stato IDLE che verifica la condizione $if\ i_start = 1$. Non appena questa condizione è vera, vengono eseguite diverse operazioni fondamentali in parallelo:

- il valore presente su i_add viene campionato e memorizzato nel registro interno $base_address$. Questo indirizzo rappresenta la locazione di memoria da cui iniziare a leggere tutti i parametri e i dati necessari per l'elaborazione.
- viene inizializzato il contatore $current_index$ con lo stesso valore, che servirà come puntatore mobile durante tutte le operazioni di lettura e scrittura.
- prepara la transizione allo stato successivo, impostando $next_state \leq READ_K1$, mentre il segnale di write enable rimane basso ($mem_we_int \leq 0$) dato che in questa fase iniziale si tratta esclusivamente di operazioni di lettura.

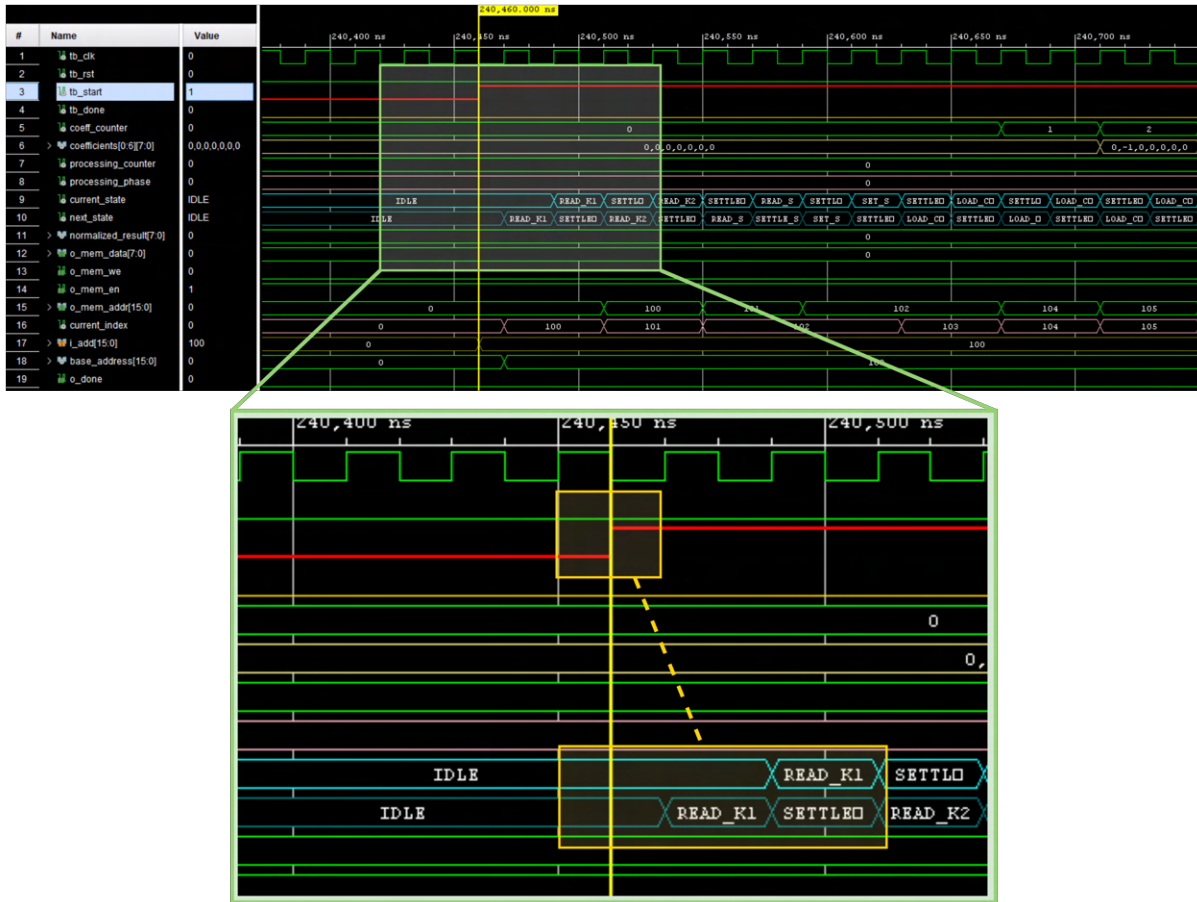


Figura 3.1: *i_start* viene alzato ad '1' e al primo fronte di salita utile del clock il processo esce dallo stato di IDLE

3.2.2. Selezione filtro

La selezione del filtro avviene dopo aver letto il byte *S* dalla memoria il cui unico bit che conta è il meno significativo: $S_{(0)}$. Quando il modulo esegue $filter_select \leq i_mem_data$, sta essenzialmente campionando questo bit e memorizzandolo in un registro interno che influenzerà tutto il resto dell'elaborazione. Qualora il valore da test bench del byte $S_{(0)}$ sia '0' il filtro sarà di ordine 3, 5 altrimenti.

Ogni elaborazione inizia da uno stato "pulito" e deterministico: quando la prima elaborazione con filtro di ordine 3 termina e il modulo entra in *DONE_STATE*, il sistema si riprepara ad una successiva esecuzione (vedi esempio *o_done*): il valore *filter_select*, così come l'array di *coefficients* e *data_window* vengono azzerati completamente così da evitare che residui della prima elaborazione possano interferire con la successiva.

Il ciclo di caricamento coefficienti procede identicamente per l'ordine 3 come per 5, ma nel secondo caso l'indirizzo di partenza di lettura del filtro è $current_index = i_add + 3 + 7$, dove 3 sono K_1, K_2, S e 7 sono gli indirizzi di $C1-C7$ del filtro di ordine 3.

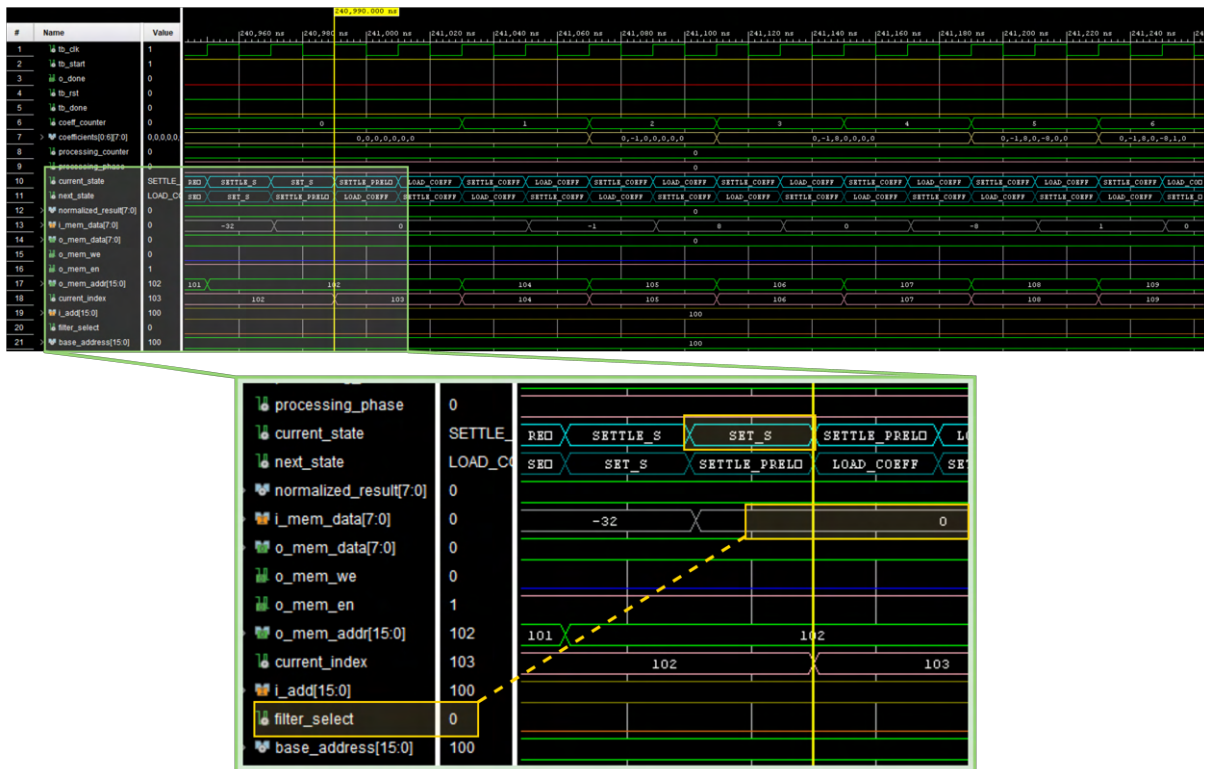


Figura 3.2: In questo caso il valore da test bench del byte S è proprio '0' quindi il filtro sarà di ordine 3.

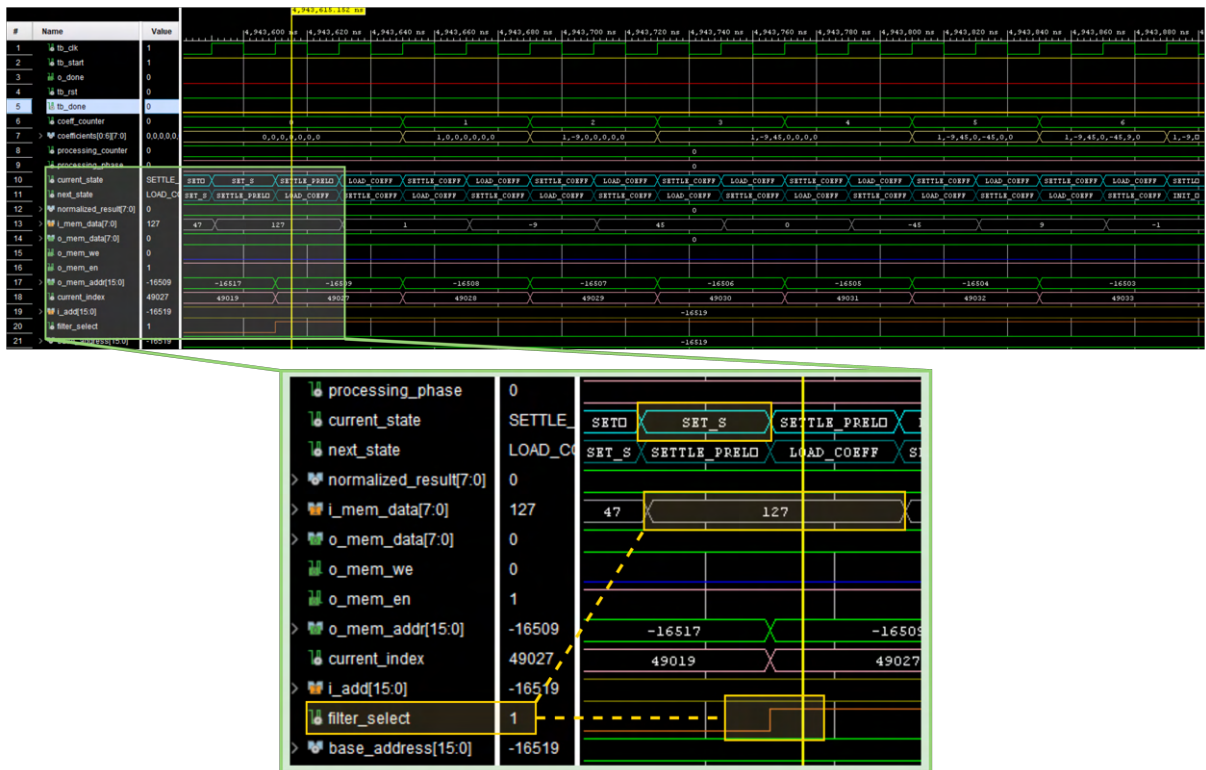


Figura 3.3: In questo caso il valore da test bench del byte $S_{(0)}$ è '1' quindi il filtro sarà di ordine 5.

Da qui in poi la FSM procederà con la gestione ed elaborazione degli stati successivi.

3.2.3. Segnale di reset nel mezzo dell'esecuzione

Il segnale *i_rst* viene alzato a '1'. Il processo *state_update_process* ha nella sua sensitivity list tre segnali: *i_clk*, *i_rst*, e *i_start*. Quando *i_rst* cambia stato, il processo viene immediatamente rieseguito e la prima istruzione del processo è “if *i_rst* = '1' then”, istruzione che ora è vera, quindi viene eseguito il ramo di reset, in particolare:

```
1   current_state <= IDLE;  
2   next_state <= IDLE;
```

forzando, così, entrambi i segnali ad *IDLE* immediatamente, prima del fronte di salita del prossimo clock interrompendo istantaneamente stato ed esecuzione corrente. Parallelamente, il processo *main_control_process* avendo anche lui *i_rst* nella sensitivity list si occupa di resettare tutti i registri interni e segnali di controllo:

In particolare, forzando *mem_en_int* a '0' se il processo si trova a metà di un'operazione di lettura (ad esempio aveva visto *mem_en* = '1' ed stava per fornire un dato), l'improvviso abbassamento di *mem_en* interrompe questa operazione e nel ciclo di clock successivo, quando il processo valuterà if *mem_en* = '0', la condizione sarà falsa, e quindi:

```
1   mem_data_out_int <= (others => '0')
```

forzando a 0 (ad alta impedenza) il bus dati. Ho quindi i registri interni dove:

- tutti i contatori sono zero
- tutti gli array sono azzerati (coefficients, data_window)
- K, base_address, filter_select azzerati

Ovvero uno stato analogo a quello in cui si troverebbe il modulo dopo un reset iniziale all'accensione

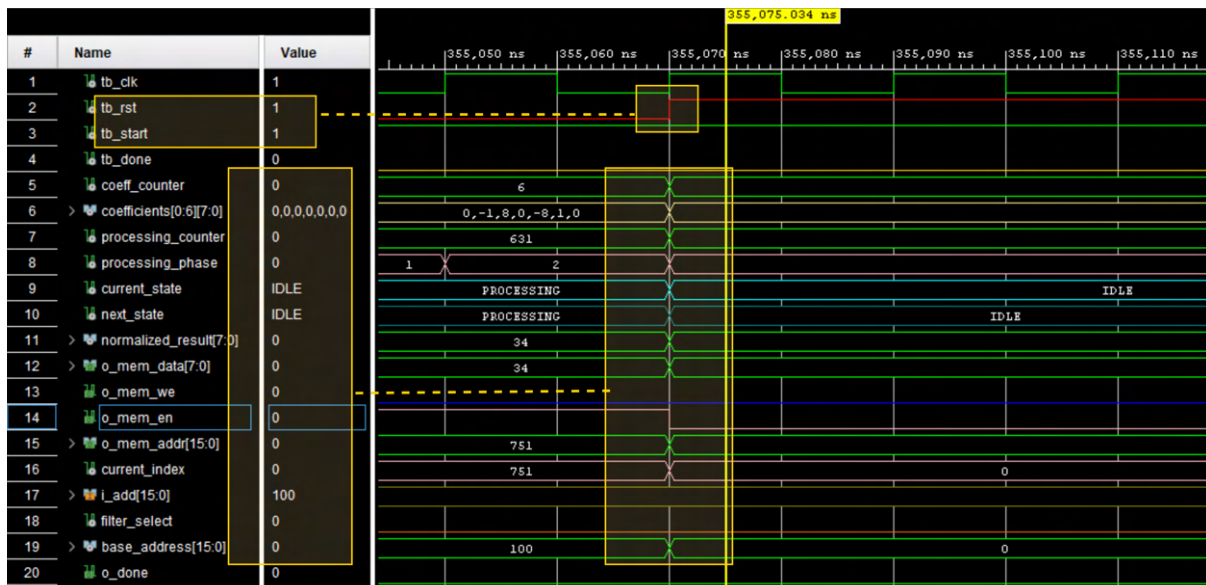


Figura 3.4: il segnale i_rst viene alzato ad '1': al successivo istante di clock tutti i segnali vengono quindi resettati

Permanenza nello Stato di Reset

Finché i_rst rimane alto, il modulo rimane “congelato” nello stato resettato. Ad ogni ciclo di clock, i processi vengono rieseguiti, ma poiché $i_rst = '1'$, vengono sempre eseguiti i rami di reset che continuano a forzare tutti i segnali ai valori iniziali. Non importa cosa succede agli altri segnali di ingresso durante questo periodo: anche se i_start dovesse essere alto, verrebbe ignorato perché il ramo if $i_rst = '1'$ ha priorità su tutto il resto.

Rilascio del Reset e Ritorno Operativo

Quando il segnale i_rst viene riabbassato, i processi vengono rieseguiti di nuovo (perché è cambiato un segnale nella loro sensitivity list), ma ora la condizione if $i_rst = '1'$ è falsa, quindi al prossimo fronte di salita di i_clk verrà ripresa la logica normale a partire dallo stato di IDLE.

3.2.4. O_done

Il ciclo di vita completo di un'elaborazione, dal punto di vista di o_done , è:

- IDLE ed elaborazione: $o_done = '0'$, il modulo sta processando
- Completamento: $o_done = '1'$, i risultati sono disponibili
- Attesa riconoscimento: $o_done = '1'$, attendendo che $START$ scenda
- Ritorno a IDLE: $o_done = '0'$ resettato come gli altri segnali per un'eventuale successiva elaborazione

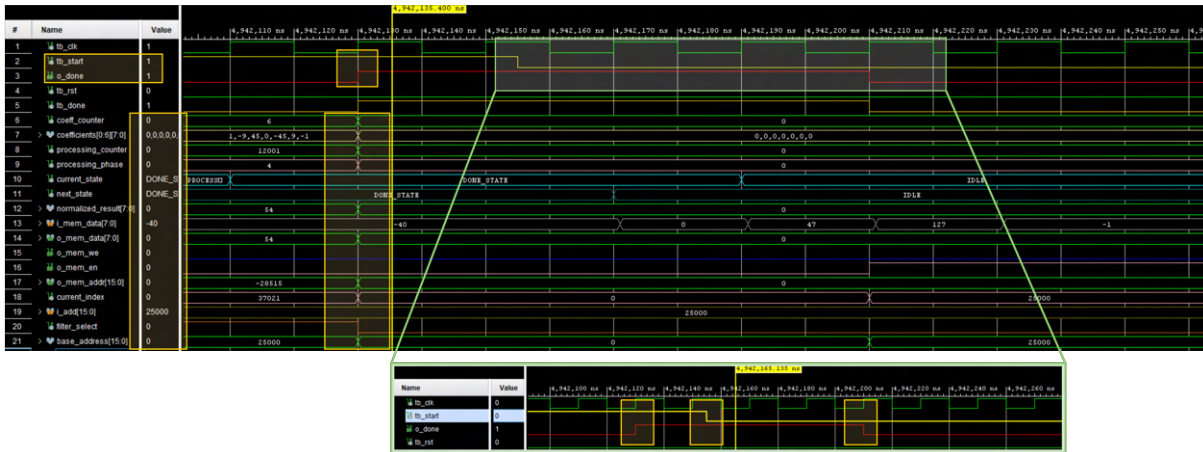


Figura 3.5: Il processo arriva a completamento e alza il segnale *o_done*, riportando i segnali allo stato iniziale. In dettaglio: 4-ways handshake

Quando *o_done* viene riportato a '0', i segnali vengono preparati per poter ricominciare un nuovo ciclo, in particolare vengono resettati:

- **dati temporanei:** *data_window*, *temp_sum*, *normalized_result* – contengono stati intermedi dell'elaborazione corrente che non saranno più necessari.
- **contatori:** *processing_counter*, *coeff_counter*, *processing_phase*, *buffer_index* – devono esser riazzerati per la prossima elaborazione
- **coefficienti:** *coefficients* – verranno ricaricati dalla memoria nella prossima run, potenzialmente diversi.
- **parametri:** *K*, *base_address*, *filter_select* – come i coefficienti, verranno riletti dalla memoria l'eventuale prossima volta.

Una parte fondamentale della preparazione è la disabilitazione completa dell'interfaccia di memoria:

```
1 mem_en_int <= '0';
2 mem_we_int <= '0';
```

. Questo garantisce che, mentre il modulo è in *DONE_STATE* e il sistema esterno sta potenzialmente leggendo i risultati dalla memoria, il modulo stesso non faccia accessi che potrebbero interferire (se il modulo continuasse ad avere *mem_en* = '1' potrebbe crearsi contesa sul bus di indirizzi o sul bus dati)

Secondo START – Indipendenza Totale dalla Prima Run

Quando *i_start* viene alzato per la seconda volta, il modulo reagisce esattamente come aveva fatto la prima volta: non c'è alcuna differenza nel comportamento, perché lo stato iniziale interno è identico.

Ogni run è quindi garantita essere identica a qualsiasi altra run con gli stessi parametri, indipendentemente da:

- Numero di run eseguite in precedenza

- Quali parametri avevano le run precedenti
- Quanto tempo è intercorso tra le run

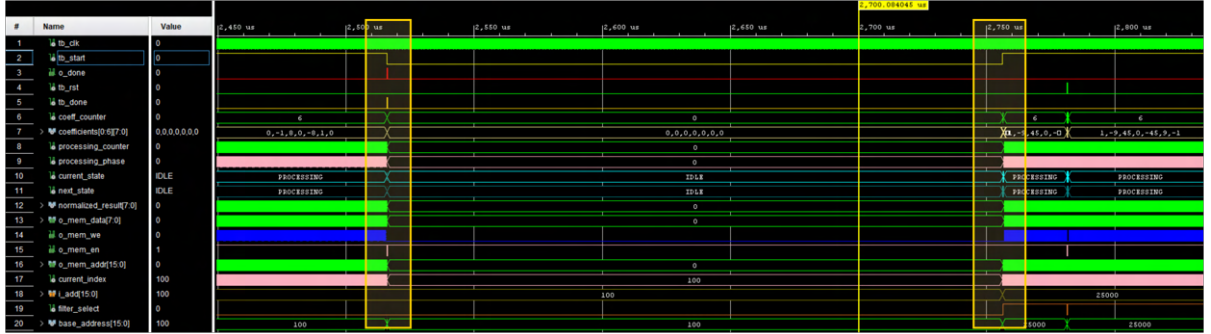


Figura 3.6: Esempio di esecuzioni multiple con ampia separazione temporale

3.2.5. Test con soli valori minimi

Il filtro di ordine 5 composto da tutti valori pari a -128, applicato a una sequenza di ingresso composta esclusivamente dal valore minimo, ovvero -128, ha dimostrato la corretta funzionalità di saturazione del modulo.

In queste condizioni estreme, il valore finale in uscita dal modulo è stato correttamente saturato, confermando che il sistema gestisce correttamente il superamento dei limiti di saturazione (**overflow**) secondo le specifiche operative previste.

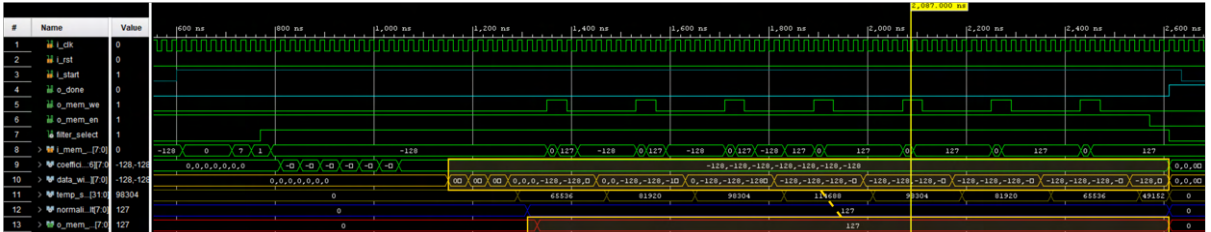


Figura 3.7: Esempio con tutti i $data_window = -128$ e tutti $coefficients = -128$

3.2.6. Test con soli valori massimi

In modo analogo, il filtro è stato testato utilizzando una sequenza di ingresso composta esclusivamente dal valore massimo rappresentabile, ovvero +127.

Anche in questo scenario limite, l'elaborazione effettuata dal filtro di ordine 5 ha condotto a un valore finale correttamente saturato. Questo risultato conferma che il modulo gestisce in modo efficace anche il superamento dei limiti inferiori di saturazione (**underflow**), garantendo la robustezza del sistema su tutto l'intervallo dinamico dei segnali di ingresso.

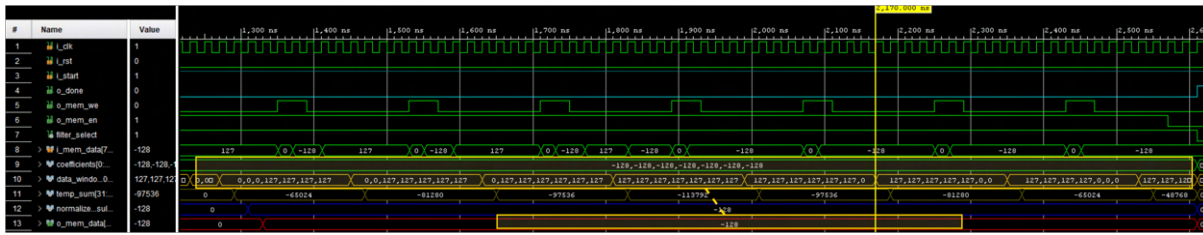


Figura 3.8: Esempio con tutti i $data_window = 127$ e tutti $coefficients = -128$

3.2.7. Test con sequenza di lunghezza massima

Considerando un i_add pari a 0 abbiamo testato il modulo sulla sequenza massima rappresentabile.

Siamo arrivati al numero considerando:

- Valore massimo rappresentabile in 16 bit: 65535.
- Sottraendo 17 bit standard.
- Dividendo il risultato per due.

$$(65535 - 17)/2 = 32759$$

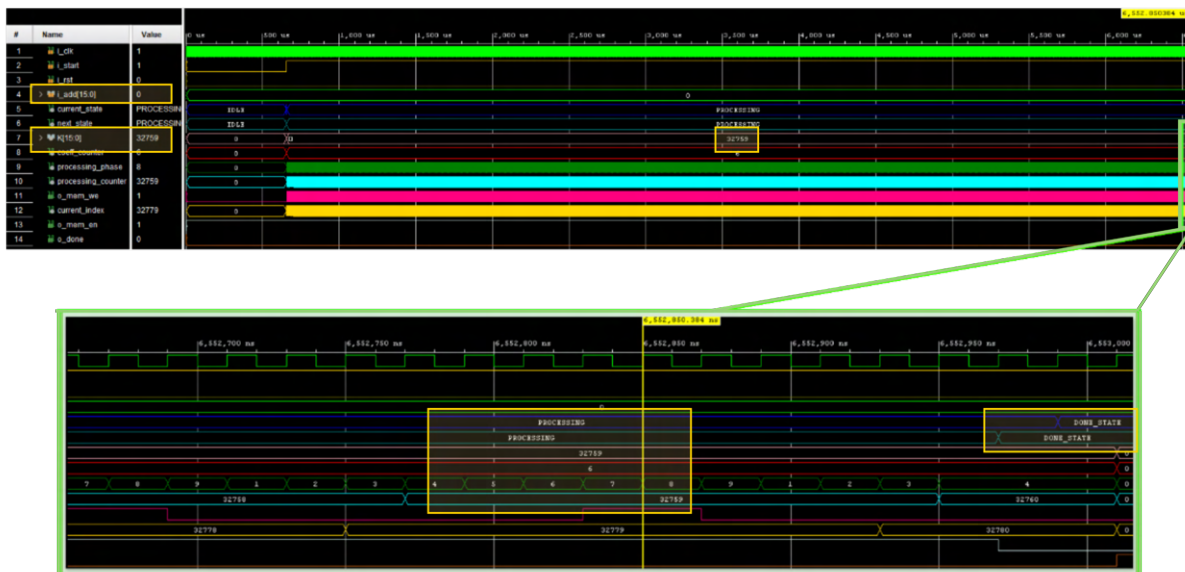


Figura 3.9: Esempio con K massimo. In dettaglio i valori finali

4 | Conclusioni

Il progetto di Prova Finale per il corso di Reti Logiche ha portato alla completa realizzazione di un'unità hardware VHDL **robusta** e **sincrona**, destinata all'elaborazione di sequenze numeriche mediante un filtro differenziale configurabile. L'obiettivo primario di fornire un componente pienamente conforme alle specifiche di interfaccia e di elaborazione è stato raggiunto con successo.

L'implementazione ha validato l'approccio architetturale basato sulla Macchina a Stati Finiti (FSM), la quale ha governato con precisione tutte le fasi operative. In particolare:

- **Gestione della Memoria**
- **Aritmetica Avanzata**
- **Verifica (Timing e Sintesi)**

L'esperienza progettuale ha consolidato la comprensione pratica dei circuiti sequenziali e delle metodologie di verifica.

In conclusione, questo lavoro ha rappresentato una significativa esperienza di progettazione digitale, fornendo un modulo VHDL che non è solo funzionalmente corretto, ma che è stato verificato in modo rigoroso sia in simulazione che in post-sintesi (timing), ponendo una solida base per l'integrazione in sistemi FPGA più complessi.

Elenco delle figure

1.1	Schema rappresentativo della sequenza	1
1.2	Rappresentazione 4-way handshake tra i_start e o_done	3
2.1	Schema della FSM di $state_update_process$	12
2.2	Rappresentazione della FSM di $processing_phase$	14
2.3	Architettura globale	15
3.1	i_start viene alzato ad '1' e al primo fronte di salita utile del clock il processo esce dallo stato di IDLE	18
3.2	In questo caso il valore da test bench del byte S è proprio '0' quindi il filtro sarà di ordine 3.	19
3.3	In questo caso il valore da test bench del byte $S_{(0)}$ è '1' quindi il filtro sarà di ordine 5.	19
3.4	il segnale i_rst viene alzato ad '1': al successivo istante di clock tutti i segnali vengono quindi resettati	21
3.5	Il processo arriva a completamento e alza il segnale o_done , riportando i segnali allo stato iniziale In dettaglio: 4-ways handshake	22
3.6	Esempio di esecuzioni multiple con ampia separazione temporale	23
3.7	Esempio con tutti i $data_window = -128$ e tutti $coefficients = -128$	23
3.8	Esempio con tutti i $data_window = 127$ e tutti $coefficients = -128$	24
3.9	Esempio con K massimo. In dettaglio i valori finali	24

Elenco delle tabelle

1.1	Interfaccia del modulo con la memoria	4
1.2	Sequenza di dati W di esempio	5
1.3	Calcolo del filtraggio con i dati d'esempio.	6
1.4	Calcolo della normalizzazione sui dati di esempio.	6
2.1	Registri e buffer principali del modulo (tutti dichiarati come signal). . . .	10
2.2	Latenza totale = $23 + 9K$ cicli di clock	14
3.1	Sintesi del Timing Report	16
3.2	Report di utilizzo delle risorse	17